# FineBank API

## Dokumentasi & Tutorial Backend

| | |
|---|---|
| Mata kuliah | Pemrograman Sisi Server |
| Dosen | Ajib Susanto, M.Kom |
| Nama | Arief Naufal Al-Azmi |
| NIM | A11.2023.15048 |
| Program Studi | Teknik Informatika (S1), Universitas Dian Nuswantoro |
| Proyek asal | FineBank Admin UI (Pemrograman Sisi Klien) - github.com/Ariefnaufal67/admin-ui |
| Repository backend | github.com/Ariefnaufal67/finebank-api (isi setelah di-push) |
| URL hosting | (isi setelah deploy, mis. https://finebank-api.up.railway.app) |

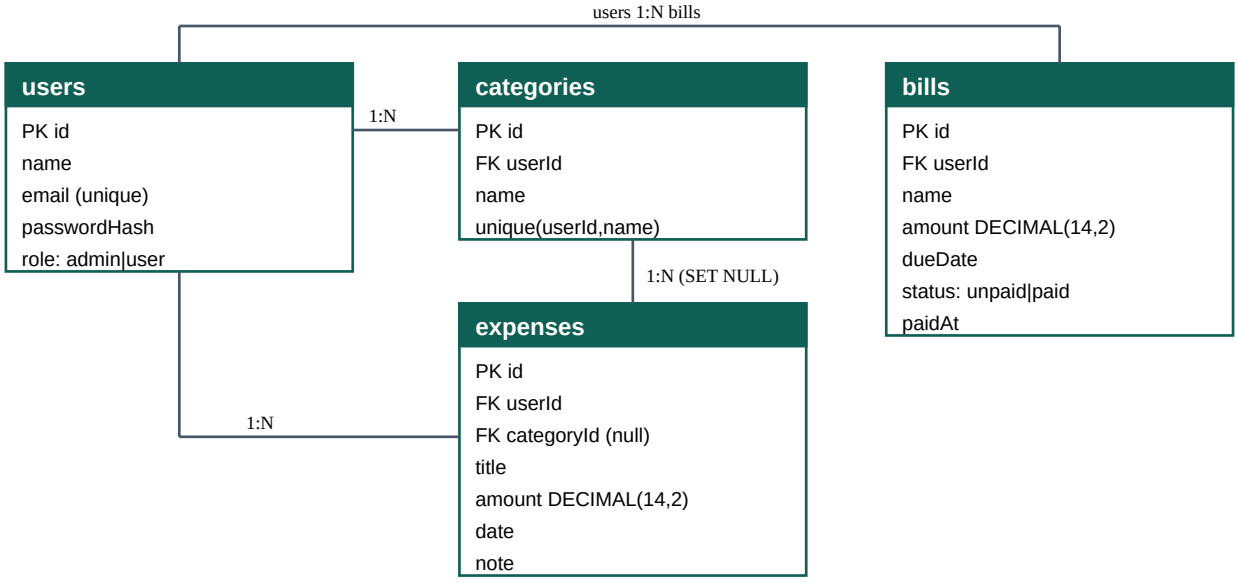
# 1. Latar Belakang & Ruang Lingkup

Pada mata kuliah Pemrograman Sisi Klien dibuat **FineBank Admin UI** (React + Vite + Tailwind + MUI) dengan halaman Expenses dan Bills. Tugas ini membangun backend pasangannya: REST API yang menyimpan data pengguna, kategori, pengeluaran (expenses), dan tagihan (bills) sehingga frontend tidak lagi bergantung pada data statis.

| Ketentuan tugas | Implementasi |
|---|---|
| Desain database, model & schema | Bab 2: 4 tabel dengan relasi, index, dan constraint (Sequelize) |
| API | Bab 4: REST JSON, 21 endpoint, format respons seragam |
| Auth | JWT Bearer + role admin/user, bcrypt |
| Throttling, pagination, filtering | express-rate-limit; page/limit/sort; q, rentang nominal & tanggal, kategori, status |
| Unit testing | 24 test (node:test + supertest), semua lulus |
| Docs | Swagger UI (/docs), OpenAPI JSON, dokumen PDF ini |
| Uji Postman | Koleksi di folder postman/ dengan test script otomatis |
| Hosting | Railway + PostgreSQL (Bab 8) |

# 2. Desain Database

Database relasional dipilih karena data keuangan bersifat terstruktur dan butuh integritas referensial. Pengembangan memakai SQLite, produksi memakai PostgreSQL; pergantian cukup lewat variabel `DATABASE_URL` karena ORM Sequelize menyamakan model.




Gambar 1. ERD FineBank API

## Aturan relasi

| Relasi | Kardinalitas | On delete |
|---|---|---|
| users - categories | 1 : N | CASCADE |
| users - expenses | 1 : N | CASCADE |

| Relasi | Kardinalitas | On delete |
|---|---|---|
| users - bills | 1 : N | CASCADE |
| categories - expenses | 1 : N (categoryId opsional) | SET NULL |

## Index & constraint

| Tabel | Aturan |
|---|---|
| users | email UNIQUE; passwordHash dikecualikan dari query default (defaultScope) |
| categories | UNIQUE (userId, name) |
| expenses | INDEX (userId, date), INDEX (categoryId); amount DECIMAL(14,2) |
| bills | INDEX (userId, dueDate), INDEX (status); status ENUM unpaid\|paid. Status "overdue" tidak disimpan, dihitung dari unpaid + dueDate < hari ini |

## 3. Struktur Proyek

```
finebank-api/
  src/
    app.js            # susunan middleware & route (createApp)
    server.js         # koneksi DB, sync, listen
    config.js  db.js  # env, koneksi Sequelize
    models/index.js   # User, Category, Expense, Bill + relasi
    schemas.js        # validasi Zod
    openapi.js        # spesifikasi Swagger
    middleware/       # auth (JWT, role), validate, error
    routes/           # auth, categories, expenses, bills, users
    utils/            # pagination, errors
    seed.js           # data contoh
  test/               # *.test.js
  postman/            # koleksi Postman
  Procfile railway.json .env.example README.md
```

Alur request: **rate limiter** → **authenticate** (verifikasi JWT, muat user) → **validate** (Zod) → route handler (query ber-scope user) → **errorHandler**. Express 5 meneruskan error dari handler async secara otomatis.

## 4. Daftar Endpoint

| Method | Path | Auth | Keterangan |
|---|---|---|---|
| POST | /api/auth/register | - | Daftar; 201, 400, 409 |
| POST | /api/auth/login | - | Login, kembalikan token; 401 |
| GET | /api/auth/me | Bearer | Profil user login |
| GET/POST | /api/categories | Bearer | Daftar / buat kategori |
| PUT/DELETE | /api/categories/:id | Bearer | Ubah / hapus kategori |
| GET | /api/expenses | Bearer | Daftar + pagination, filter, sort |
| GET | /api/expenses/summary | Bearer | Total & ringkasan per kategori |
| POST | /api/expenses | Bearer | Tambah expense |
| GET/PUT/DELETE | /api/expenses/:id | Bearer | Detail / ubah / hapus |
| GET | /api/bills | Bearer | Daftar + pagination, filter status, sort |
| POST | /api/bills | Bearer | Tambah bill |
| GET/PUT/DELETE | /api/bills/:id | Bearer | Detail / ubah / hapus |
| POST | /api/bills/:id/pay | Bearer | Tandai lunas (isi paidAt) |
| GET | /api/users | Admin | Daftar user + pagination, q, role |
| DELETE | /api/users/:id | Admin | Hapus user |
| GET | /health, /docs, /openapi.json | - | Health check & dokumentasi |

### Format respons

```
// sukses
{ "success": true, "data": [...], "meta": { "page": 1, "limit": 10, "total": 25,
                                             "totalPages": 3, "hasNext": true, "hasPrev": false } }
// gagal
{ "success": false, "message": "Validasi gagal",
  "errors": [ { "field": "amount", "message": "Nominal harus lebih dari 0" } ] }
```

# 5. Auth, Throttling, Pagination & Filtering

## 5.1 Autentikasi

Password di-hash dengan bcrypt (cost 10). Login mengembalikan JWT (HS256, masa berlaku 1 hari) berisi id dan role. Middleware `authenticate` memverifikasi token dan memuat user dari database, sehingga akun yang sudah dihapus langsung kehilangan akses. Role **user** hanya melihat data miliknya (query selalu ber-scope `userId`, data orang lain menghasilkan 404); role **admin** melihat seluruh data dan dapat mengelola user. Registrasi publik selalu membuat role user; admin dibuat lewat seed/DB.

```
POST /api/auth/login
{ "email": "arief@finebank.test", "password": "password123" }
-> 200 { "success": true, "data": { "user": {...}, "token": "eyJhbGci..." } }

GET /api/expenses
Authorization: Bearer eyJhbGci...
```

## 5.2 Throttling

| Cakupan | Batas default | Env |
|---|---|---|
| Semua /api/* | 100 request / 15 menit / IP | RATE_LIMIT_MAX, RATE_LIMIT_WINDOW_MS |
| /api/auth/login & /register | 10 request / 15 menit / IP | RATE_LIMIT_AUTH_MAX |

Kelebihan batas dijawab **429** dengan pesan JSON dan header standar `RateLimit-*`. Pada uji langsung, 12 percobaan login berturut-turut menghasilkan 401 sebanyak 9 kali lalu 429 mulai percobaan ke-10 (satu request login awal ikut terhitung).

## 5.3 Pagination, filter, dan sort

| Parameter | Endpoint | Contoh / aturan |
|---|---|---|
| page, limit | semua list | default 1 dan 10; limit maksimum 100 |
| sort | expenses, bills | sort=-amount (desc), sort=date (asc); hanya kolom yang diizinkan |
| q | expenses, bills, users | pencarian teks pada title / name |
| categoryId | expenses | categoryId=2 |
| minAmount, maxAmount | expenses | rentang nominal |
| from, to | expenses (date), bills (dueDate) | format YYYY-MM-DD, divalidasi |
| status | bills | unpaid \| paid \| overdue |

```
GET /api/expenses?q=kopi&minAmount=10000&from=2026-10-01&sort=-amount&page=1&limit=5
GET /api/bills?status=overdue
```

Sort memakai whitelist kolom sehingga nilai sembarang (mis. passwordHash) diabaikan. Query memakai parameter terikat Sequelize, bukan penggabungan string, sehingga aman dari SQL injection.

# 6. Unit Testing

Test memakai `node:test` bawaan Node.js dan `supertest`, dengan SQLite in-memory yang direset sebelum tiap test. Jalankan `npm test`.

| Berkas | Yang diuji |
|---|---|
| pagination.test.js (3) | parsePagination, parseSort, buildMeta (unit murni) |
| auth.test.js (7) | register, isi klaim JWT, validasi, duplikat email, login benar/salah, /me, throttling login & global |
| expenses.test.js (8) | wajib login, CRUD, validasi, pagination meta, filter + sort, isolasi data & akses admin, categoryId milik orang lain, summary |
| bills.test.js (6) | isOverdue, bayar bill, filter status, pagination, akses khusus admin, 404 & JSON rusak |

### Hasil eksekusi:

```
# tests 24
# pass 24
# fail 0
```

# 7. Pengujian dengan Postman

Impor `postman/FineBank.postman_collection.json`, set variabel `baseUrl` (lokal atau URL hosting), lalu jalankan dengan Collection Runner. Folder berurutan: Auth, Categories, Expenses, Bills. Token dan id disimpan otomatis ke variabel koleksi lewat test script, dan tiap request memeriksa status code (termasuk kasus 400 dan 401). Jalankan `npm run seed` dulu agar request "Login (seed)" berhasil.

### Bukti uji langsung terhadap server yang berjalan (curl):

```
GET /api/expenses?limit=2&sort=-amount   -> 200, 2 data, meta {total:4, totalPages:2, hasNext:true}
GET /api/bills?status=overdue            -> 200, hanya bill "Listrik" (isOverdue:true)
GET /docs/                               -> 200 (Swagger UI)
12x POST /api/auth/login salah           -> 401 x9, lalu 429
```

Lampirkan tangkapan layar Collection Runner pada bagian ini sebelum dikumpulkan.

## 8. Hosting (Railway + PostgreSQL)

| Langkah | Detail |
|---|---|
| 1 | Push proyek ke GitHub (.env dan data/ sudah di .gitignore). |
| 2 | Railway: New Project → Deploy from GitHub repo → pilih repo finebank-api. |
| 3 | Tambah database PostgreSQL pada project yang sama; Railway menyediakan DATABASE_URL. |
| 4 | Variables pada service API: JWT_SECRET (string acak panjang), CORS_ORIGIN = https://admin-ui-chi-rust.vercel.app, DATABASE_URL (referensi dari service Postgres), DATABASE_SSL=true bila koneksi menolak tanpa SSL. |
| 5 | Generate Domain; cek /health lalu /docs. Tabel dibuat otomatis oleh sequelize.sync() saat pertama start. |
| 6 | Opsional: jalankan seed sekali lewat Railway shell (npm run seed). Perhatian: seed memakai force:true dan menghapus isi tabel. |

Healthcheck diatur pada `railway.json`. Alternatif gratis: Render + Neon/Supabase dengan variabel yang sama.

## 9. Integrasi dengan FineBank Admin UI

Frontend sebelumnya memakai backend JWT milik mata kuliah (endpoint /login, /expenses, /bills; error berbentuk {msg}). Integrasi dilakukan hanya di `src/services/authService.jsx`, sehingga halaman, komponen, dan tes Cypress tidak berubah:

| Penyesuaian | Alasan |
|---|---|
| URL dari VITE_API_URL (default http://localhost:3000) | Dapat diganti ke URL hosting tanpa ubah kode; set juga di Environment Variables Vercel |
| POST /api/auth/login, hasilnya dikembalikan sebagai { refreshToken } | Signin.jsx tetap memakai properti refreshToken |
| JWT berisi name, email, role | authContext memakai jwtDecode, MainLayout menampilkan user.name |
| Error {message, errors} dipetakan ke { msg } | Komponen UI membaca err.msg |
| Expenses: limit=100, category objek diratakan menjadi nama string | API berhalaman (default 10); CardExpenseComparison mengharapkan category berupa string |
| Bills: status=unpaid, sort=dueDate | Kartu Upcoming Bill hanya menampilkan tagihan yang belum dibayar |
| Nama kategori seed: Food, Transport, Entertainment | Cocok dengan peta ikon kategori di frontend |

```
// authService.jsx (ringkas)
const API_URL = (import.meta.env?.VITE_API_URL || "http://localhost:3000");
export const loginService = async (email, password) => {
  const res = await axios.post(`${API_URL}/api/auth/login`, { email, password });
  return { refreshToken: res.data.data.token };
};
```

Diverifikasi dengan menjalankan service frontend yang asli terhadap server backend: login, jwtDecode (name dan role terbaca), 4 expenses ber-kategori string, 2 bills unpaid, serta pesan error login dan register terpetakan ke { msg }. Build Vite membaca VITE_API_URL dengan benar.

## 10. Keamanan & Batasan

Terapan: helmet, CORS terkonfigurasi, bcrypt, batas ukuran body 100 KB, validasi semua input, scope data per user, secret lewat environment. Batasan yang diketahui: belum ada refresh token / logout

server-side (token berlaku 1 hari), rate limit disimpan di memori proses sehingga tidak dibagi antar instance, dan skema dibuat dengan `sync()` tanpa migrasi bertahap.

## 11. Kesimpulan

Backend FineBank memenuhi seluruh poin ruang lingkup tugas: desain database relasional, API REST ber-auth, throttling, pagination dan filtering, 24 test otomatis, dokumentasi Swagger, pengujian Postman, dan konfigurasi hosting.